

Digital design intern ADI

FPGA assignment 1 3bit counter

Made by:

Hagar Emadeldin Fathy Emara

Submitted to:

Eng/ Fatma Ali

The codes:

```
A_assign_1.srcs > sources_1 > new > V 3_bit_counter.v > ...  
module counter_3bit (  
    input wire    clk,  
    input wire    rst_n,  
    input wire    tick,  
    output reg [2:0] led_out  
);  
  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n)  
            led_out <= 3'b000;  
        else if (tick)  
            led_out <= led_out + 1'b1;  
        end  
endmodule
```

```

module clock_divider(
    input wire clk,
    input wire rst_n,
    output reg tick
);

    parameter CLK_FREQ_HZ = 8000000;
    parameter TARGET_HZ    = 1;
    localparam MAX_COUNT = (CLK_FREQ_HZ / TARGET_HZ) - 1;

    reg [22:0] count;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            count <= 0;
            tick  <= 1'b0;
        end else if (count == MAX_COUNT) begin
            count <= 0;
            tick  <= 1'b1;
        end else begin
            count <= count + 1'b1;
            tick  <= 1'b0;
        end
    end

endmodule

```

lab_assign_1.srcs > sources_1 > new > top.v

```
1 module top (  
2     input wire      osc_clk,  
3     input wire      btn_rst,  
4     output wire [2:0] led  
5 );  
6  
7     wire clk_8mhz;  
8     wire pll_locked;  
9     wire rst_n = pll_locked & ~btn_rst;  
10  
11     wire tick;  
12  
13     clk_wiz_0 u_pll ( .clk_in1 (osc_clk), .clk_out1 (clk_8mhz), .reset(btn_rst), .locked (pll_locked));  
14  
15     clock_divider #( .CLK_FREQ_HZ (8_000_000), .TARGET_HZ (1)) u_div ( .clk (clk_8mhz), .rst_n (rst_n), .tick (tick));  
16  
17     counter_3bit u_cnt ( .clk (clk_8mhz), .rst_n (rst_n), .tick (tick), .led_out (led));  
18  
19 endmodule
```

lab_assign_1.srcs > sources_1 > new > test.v

```
1 timescale 1ns/1ps  
2  
3 module tb_top;  
4  
5     reg      osc_clk;  
6     reg      btn_rst;  
7     wire [2:0] led;  
8  
9     top uut (  
10         .osc_clk (osc_clk),  
11         .btn_rst (btn_rst),  
12         .led (led)  
13     );  
14  
15     always #5 osc_clk = ~osc_clk;  
16  
17     initial begin  
18         osc_clk = 0;  
19         btn_rst = 1;  
20         #1000;  
21         btn_rst = 0;  
22  
23         $display("Waiting for PLL lock and first counter increments...");  
24     end  
25  
26     initial begin  
27         $monitor("Time=%0t ns | led=%b", $time, led);  
28     end  
29  
30     initial begin  
31         #5_000_000_000;  
32         $display("Simulation complete.");  
33         $stop;  
34     end  
35  
36 endmodule
```

After creating adding the PLL and choosing the clock to be 8 MHZ output, I made an initialization for the PLL IP in the top module, and then made the constraints related to the 8MHZ clock named `osc_clk`, but the design cannot run with the 8 MHZ, as it took the input of the tick clock and the condition was made that the the tick signal is not active until the certain no of cycles pass, so we skip cycles to get the frequency and the edge we need.

```
_assign_1.srscs > constrs_1 > new > cons.xdc
# -----
# Clock Source - Bank 13 (100 MHz oscillator -> PLL clk_in1)
# -----
set_property PACKAGE_PIN Y9 [get_ports {osc_clk}];
set_property IOSTANDARD LVCMOS33 [get_ports {osc_clk}];
create_clock -name osc_clk -period 10.000 [get_ports {osc_clk}];

# -----
# User DIP Switch - Bank 35 (reset input)
# -----
set_property PACKAGE_PIN F22 [get_ports {btn_rst}];      # "SW0"

# -----
# User LEDs - Bank 33 (3-bit counter output)
# -----
set_property PACKAGE_PIN T22 [get_ports {led[0]}];      # "LD0"
set_property PACKAGE_PIN T21 [get_ports {led[1]}];      # "LD1"
set_property PACKAGE_PIN U22 [get_ports {led[2]}];      # "LD2"

# -----
# Timing exceptions
# -----
set_false_path -to [get_ports {led[*]}]
set_false_path -from [get_ports {btn_rst}]

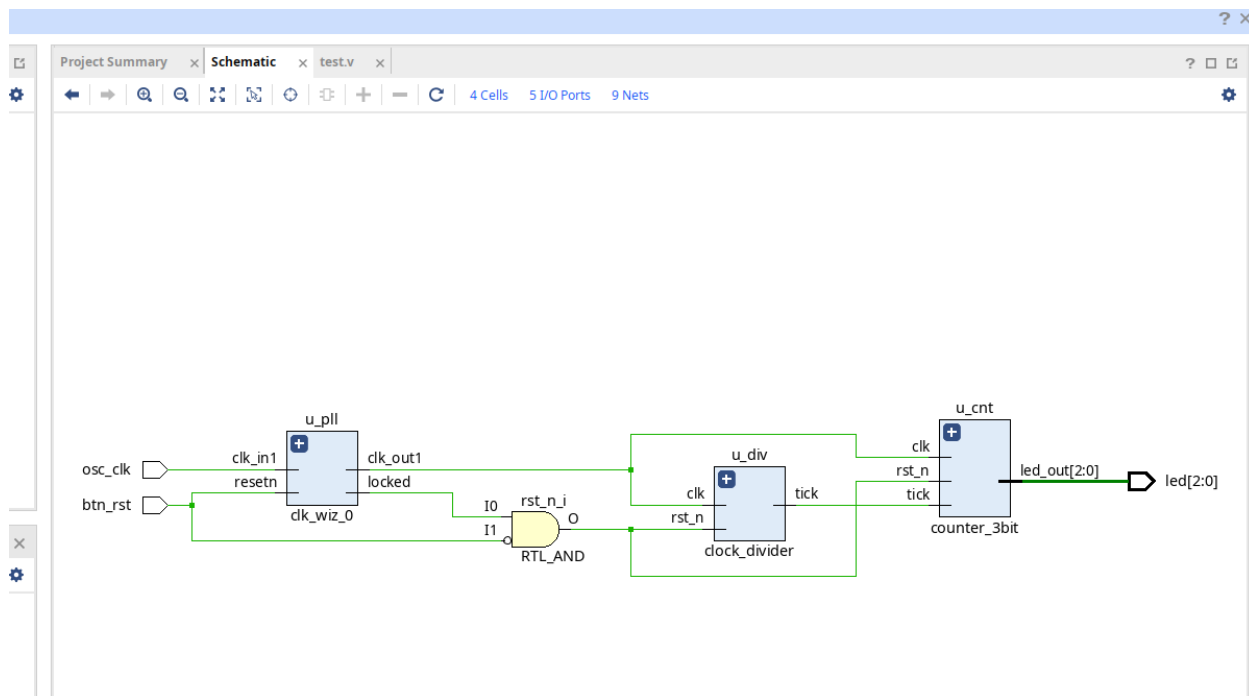
# -----
# IOSTANDARD Constraints (bank-wide, per Zedboard-Master.xdc)
# -----
set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 33]]; # LEDs - fixed 3.3V

# Bank 34 unused in this design - comment out entirely to avoid critical warning
# set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 34]];

set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 35]]; # reset switch

set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 13]]; # clock input - fixed 3.3V
```

After elaboration



as wshown in the waveform, when the tick signal is on the count starts to count up one more

